ENGENHARIA DE SOFTWARE | ENGENHARIA DE REQUISITOS

Professora: Kadidja Valéria

Turma: B

Alunos: Guilherme Valadao Pereira
Davi Lemes de Paula
Gianne Clarice Silva Alencar

4. Etapa 1 — Leitura e identificação

Elemento identificado: Selecionar opção de tutorial
Evidência no artigo: "O software deverá permitir ao usuário selecionar a opção de tutorial"
Classificação: Requisito funcional
Elemento identificado: Alternativas textuais
Evidência no artigo: "Oferecer alternativas em texto para conteúdos do jogo que não forem textuais"
Classificação: Requisito de acessibilidade
Elemento identificado: Controle de Áudio
Evidência no artigo: "Oferecer opções para desativar/pausar/diminuir volume de sons como música de fundos, efeitos sonoros etc."
Classificação: Requisito de acessibilidade
Elemento identificado: Acessível por teclado
Evidência no artigo: "Permitir que o jogo possa ser completamente operado por meio do teclado"
Classificação: Requisito de acessibilidade
Elemento identificado: Sem limite de tempo
Evidência no artigo: "O jogador não precisará agir de forma rápida/ágil em nenhuma interação com o jogo"
Classificação: Requisito de acessibilidade

1 Sim faz parte pois o vilão toma dano e é derrotado quanto mais respostas certas o jogador der

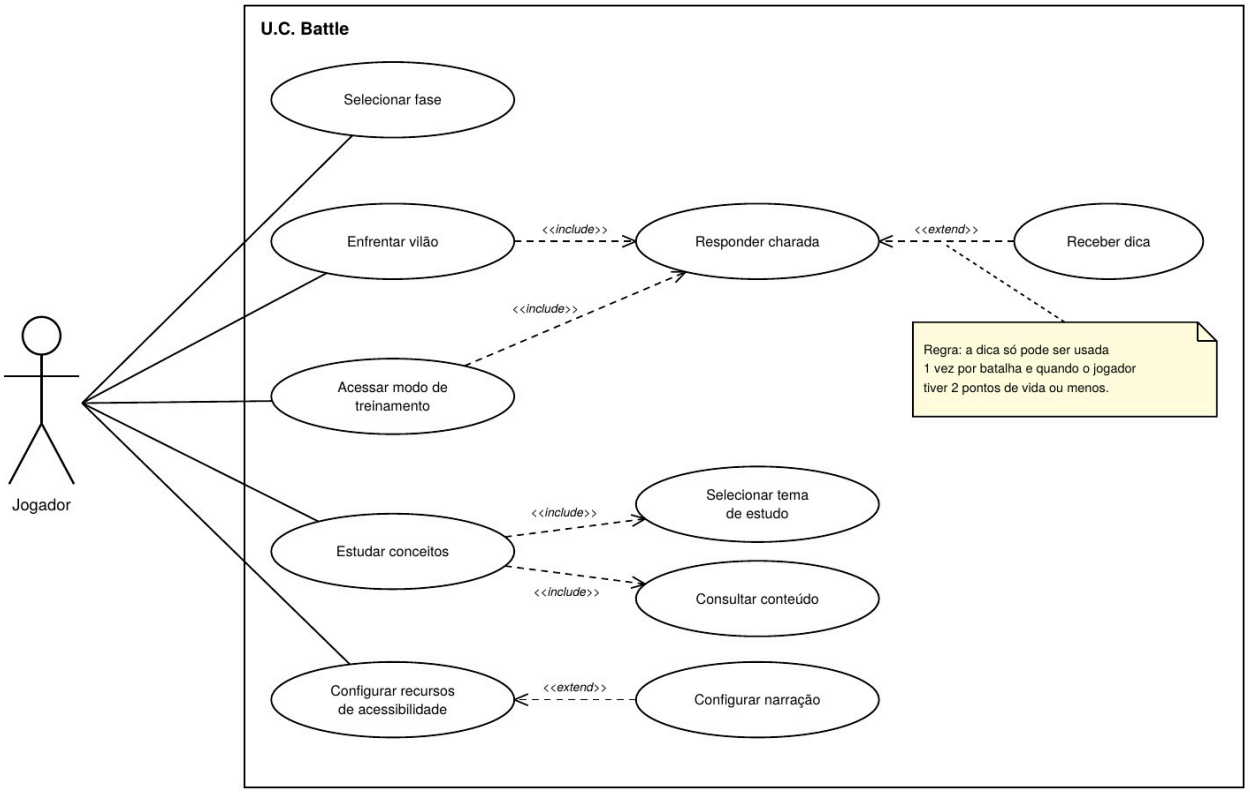
2 O jogador só pode usar dica uma única vez por batalha e se tiver 2 pontos de vida ou menos

3 É uma funcionalidade independente, e não uma etapa da batalha.

4 Sim, o modo de treinamento reutiliza a prática do combate, mas o jogador não sofre penalidade

5 O estudo lista e implementa um conjunto de 21 requisitos de acessibilidade divididos em diferentes diretrizes, que incluem:
Alternativas textuais e narração de conteúdo textual.
Informações e relações e sequência significativa.
Características sensoriais.
Contraste mínimo e contraste não textual.
Controle de áudio .
Utilização de cores.
Conteúdo em foco por mouse ou teclado e acessibilidade completa por teclado.
Sem limite de tempo nas interações.
Página com título e rótulo em nome acessível.
Idioma da página, itens em foco e navegação consistente.
Progressão natural de dificuldade.
Identificação do erro, rótulos ou instruções e modo tutorial. '

5. Etapa 2 — Construção do Diagrama



6. Etapa 3 — Matriz de rastreabilidade

ID	Requisito do artigo	Caso de uso correspondente	No diagrama?
RF01	O software deverá permitir ao usuário selecionar a opção de estudar conceitos	Estudar conceitos	Sim
RF02	O software deverá permitir ao usuário escolher uma opção de tema para poder estudá-lo	Selecionar tema de estudos	Sim
RF03	O software deverá permitir ao usuário selecionar uma fase	Selecionar fase	Sim
RF04	O software deverá trocar de charada após o usuário escolher uma resposta	Responder charada	Sim
RF05	O software deverá decrementar um ponto de vida do vilão quando o usuário acertar uma charada	Enfrentar vilão	Sim

7. Checklist de validação

- ☑ ~~Os atores representam papéis externos ao sistema.~~
- ☑ ~~Os casos de uso foram escritos com verbo no infinitivo.~~
- ☑ ~~A fronteira do sistema está identificada.~~
- ☐ Cada associação representa uma interação real.
- ☑ ~~O <<include >> representa um comportamento obrigatório e reutilizado.~~
- ☐ O <<extend >> representa um comportamento opcional ou condicionado.
- ☑ ~~As regras de negócio não foram representadas incorretamente como atores.~~

- [ ] Os requisitos selecionados estão rastreados no diagrama.
- [x] ~~Pelo menos uma necessidade de acessibilidade foi considerada.~~
- [x] ~~O diagrama está legível e sem elementos desconectados.~~

Justificativa:

A modelagem priorizou a clareza nas interações do jogador, separando os fluxos de combate, estudo e acessibilidade. Foram empregadas relações de inclusão (`<<include>>`) para comportamentos obrigatórios e de extensão (`<<extend>>`) para a regra restritiva de dicas. Desse modo, o diagrama reflete com precisão tanto a lógica do jogo quanto os requisitos de acessibilidade definidos no artigo.

Questão de encerramento:

Que problemas podem surgir durante o desenvolvimento quando um requisito funcional não possui correspondência clara no modelo UML?

Os problemas que podem surgir são: falhas graves de comunicação e alinhamento entre a análise e a implementação. Isso gera ambiguidades para os desenvolvedores, dificulta a rastreabilidade (impedindo saber com certeza se o requisito foi atendido), aumenta o risco de esquecimento de funcionalidades durante a codificação e gera retrabalho na fase de testes.